

Binaire et hexadécimal

Comment les nombres sont-ils représentés dans un ordinateur ?

$$(1100'1010'1111'1110)_2 = (51'966)_{10} = (\text{CAFE})_{16}$$

Représentation des nombres dans un ordinateur

$$(255)_{10} = (1111'1111)_2 = (FF)_{16}$$

1

Compter

Bases et notations

2

Convertir

Binaire $\leftrightarrow$ décimal

3

Calculer

Addition,
multiplication, division

4

Nombres négatifs

Le complément à
deux

5

Hexadécimal

Le binaire en plus
compact

Sommaire

1. Compter

2. Binaire $\leftrightarrow$ décimal

3. Addition

4. Multiplier et diviser

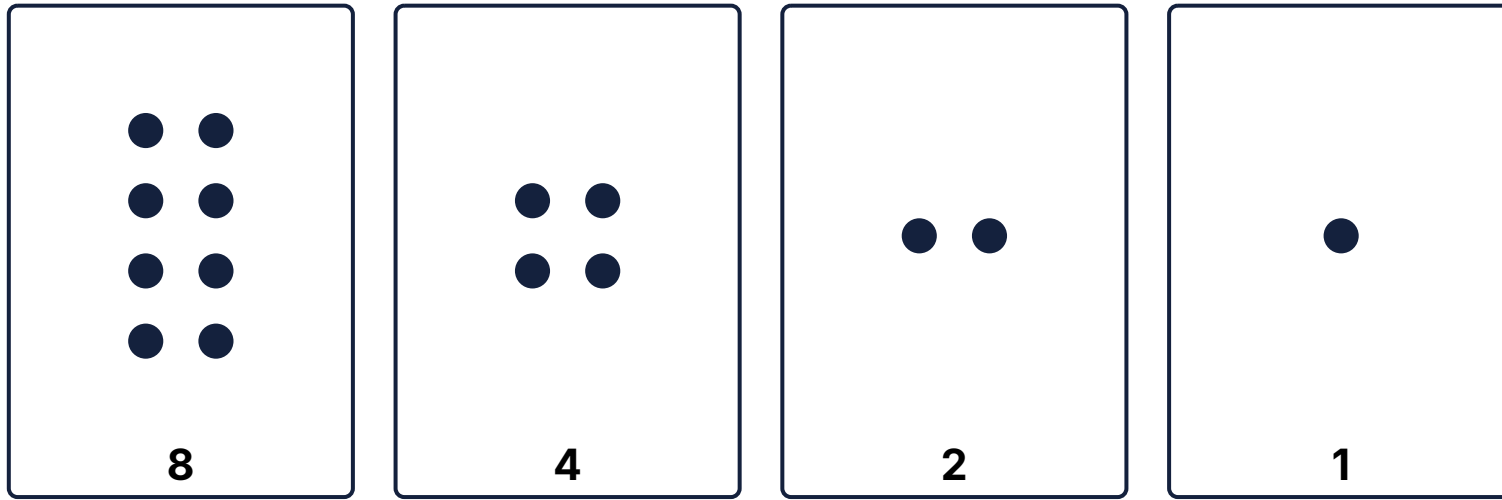
5. Nombres négatifs

6. Hexadécimal

Activité débranchée

1. Compter

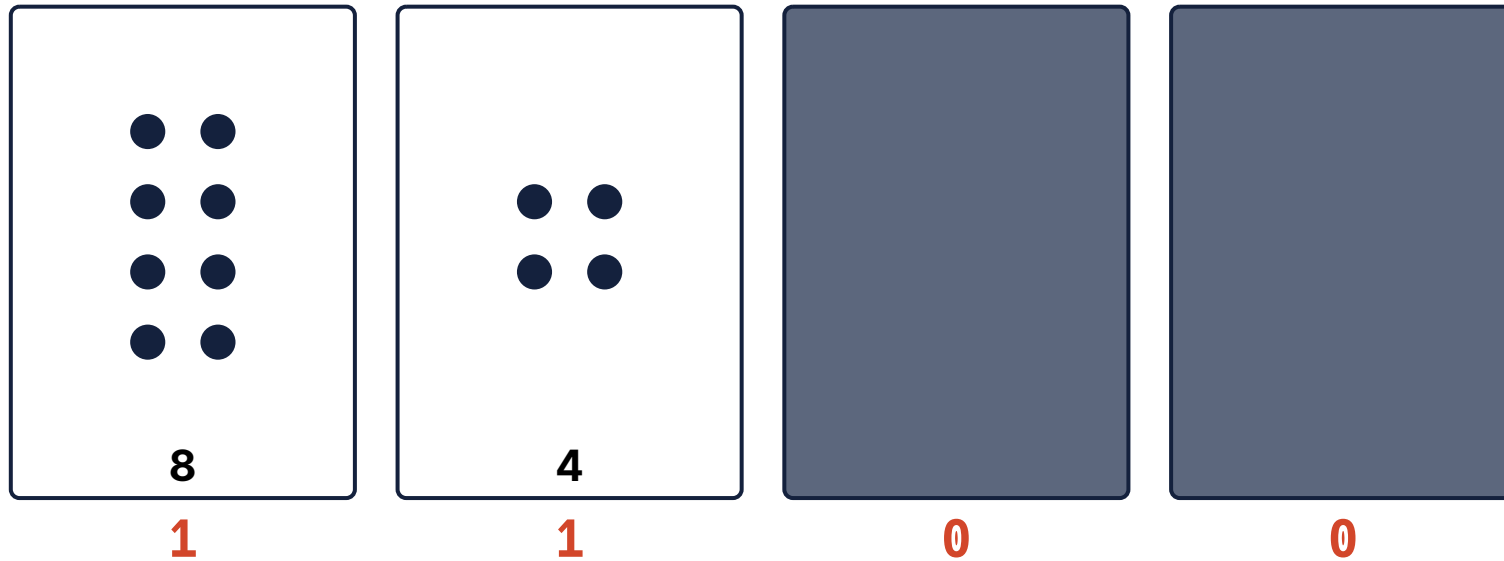
Disposer les cartes dans l'ordre, la plus grande à gauche.



Activité débranchée

1. Compter

On code les cartes avec des 0 (carte de dos) et des 1 (les points sont visibles)



$$1100 \rightarrow 8 + 4 = 12$$

Activité débranchée

1. Compter

Au tableau et de vive voix

Compter en base 10, compter en base 2

1. Compter

En quelle base compte-t-on habituellement ? Avec quels symboles ?

Décimal · base 10

Un nombre entier décimal s'écrit uniquement avec les dix symboles

0 1 2 3 4 5 6 7 8 9

Exemple : **124'509**

Binaire · base 2

Un nombre entier binaire s'écrit uniquement avec les deux symboles

0 1

Exemple : **1'0011**

D'autres bases

1. Compter

On peut compter dans n'importe quelle base, et selon les cultures, on l'a fait :

base 8

Les Yuki, en Californie : les espaces entre les doigts

base 20

Les Mayas ; en français : quatre-vingts

base 60

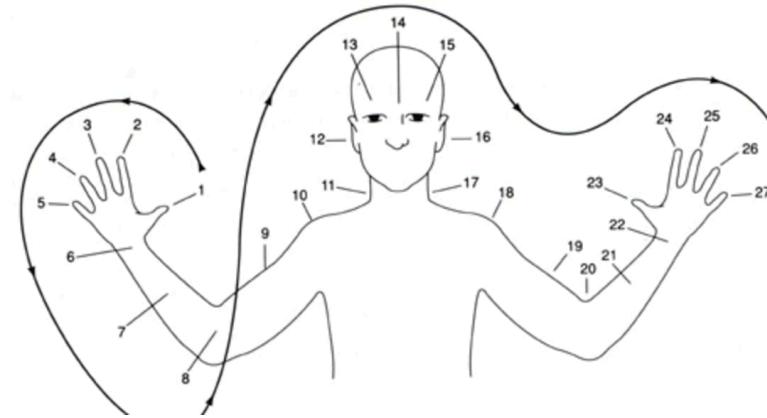
Les Babyloniens, en Mésopotamie

base 27

Les Oksapmin, en Papouasie-Nouvelle-Guinée : sur le corps

Activité · 4 min

Quelles bases autres que 10 utilisons-nous dans la vie de tous les jours ?



Compter jusqu'à 27 sur le corps (Oksapmin)

À retenir : la base 10 n'est qu'un choix, sans doute dicté par nos dix doigts. Toutes les bases permettent d'écrire les mêmes nombres.

Compter sans base

1. Compter

On ajoute une coche pour chaque nouvel objet compté. Pour savoir où on en est, il faut tout recompter depuis le début.



L'os de Lebombo, entre l'Afrique du Sud et l'Eswatini : il y a 44'000 ans, l'un des plus anciens objets de comptage connus.

Une autre notation : les chiffres romains

1. Compter

C'est à peu près de la base 10, mais avec une autre notation.

I

1

V

5

X

10

L

50

C

100

D

500

M

1000

On additionne les symboles

LXVII = $50 + 10 + 5 + 1 + 1 =$
67

**Un symbole plus petit placé
avant un plus grand se
soustrait**

CDXII = $(500 - 100) + 10 + 1$
 $+ 1 =$ **412**

**Un symbole ne se répète
pas plus de trois fois**

40 = **XL**, et non **XXXX**

Activité · 4 min

Calculer **LXXXVII** + **XI**, en chiffres romains. Comment avez-vous fait ? Pourquoi ?

La notation positionnelle

1. Compter

La valeur d'un chiffre dépend de sa position : son **poids**, une puissance de la base.

En décimal · les poids sont des puissances de 10

	1000	100	10	1
poids	10^3	10^2	10^1	10^0
chiffre	7	2	4	8

$$(7248)_{10} = 7 \times 1000 + 2 \times 100 + 4 \times 10 + 8 \times 1$$

En binaire · les poids sont des puissances de 2

	32	16	8	4	2	1
poids	2^5	2^4	2^3	2^2	2^1	2^0
chiffre	1	0	0	1	1	0

$$(100110)_2 = 32 + 4 + 2 = (38)_{10}$$

Le bit

La plus petite unité d'information : **0** ou **1**.

Contraction de l'anglais *binary digit*, chiffre binaire.

L'octet · *byte* en anglais

Huit bits.

Les nombres de ce cours tiennent souvent sur un octet.

Exemples

- **1001'0001** est un octet ; il vaut **145** en décimal.
- **111** aussi : on complète à gauche avec des 0, **0000'0111**.

On groupe les bits par 4 avec une apostrophe, pour les lire plus facilement.

Sommaire

1. Compter

2. Binaire $\leftrightarrow$ décimal

3. Addition

4. Multiplier et diviser

5. Nombres négatifs

6. Hexadécimal

La grille de conversion

2. Binaire ↔ décimal

Une grille pour convertir dans les deux sens : chaque colonne a un poids.

8 bits, soit un octet

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	0	1	1	19	16 + 2 + 1

Binaire → décimal, par exemple **1'0011** :

1. Écrire chaque bit sous son poids, en partant de la droite.
2. Dans « Calculs », noter les poids des bits à 1.
3. Les additionner : c'est la valeur en décimal.

Le plus grand nombre sur un octet

2. Binaire ↔ décimal

Activité · 3 min

Quel est le plus grand nombre qu'on peut écrire avec 8 bits ?

Le plus grand nombre sur un octet

2. Binaire ↔ décimal

Quel est le plus grand nombre qu'on peut écrire avec 8 bits ?

Tous les bits à 1

$$\mathbf{1111'1111} = 128 + 64 + 32 + 16 \\ + 8 + 4 + 2 + 1 = \mathbf{255}$$

Le nombre suivant

$$\mathbf{1'0000'0000} = \mathbf{256} : \text{un } 9^{\text{e}} \text{ bit,} \\ \text{de poids 256. Donc } 256 - 1 = \\ \mathbf{255}.$$

Les combinaisons

Chaque bit a 2 valeurs : $2 \times 2 \times \dots$
 $\times 2 = 2^8 = 256$ combinaisons,
pour les nombres de 0 à **255**.

À retenir : avec n bits, on écrit 2^n nombres différents, de 0 à $2^n - 1$.

Décimal → binaire : Par soustractions successives

2. Binaire ↔ décimal

Convertir **76** en binaire, en partant des poids les plus forts :

Poids			Bit	
128	rentre dans 76 ?	non	0	
64	rentre dans 76 ?	oui	1	$76 - 64 = 12$
32	rentre dans 12 ?	non	0	
16	rentre dans 12 ?	non	0	
8	rentre dans 12 ?	oui	1	$12 - 8 = 4$
4	rentre dans 4 ?	oui	1	$4 - 4 = 0$
2	rentre dans 0 ?	non	0	
1	rentre dans 0 ?	non	0	

8 bits, soit un octet

128	64	32	16	8	4	2	1
0	1	0	0	1	1	0	0

Résultat : **76** s'écrit **100'1100** en binaire.

Le 0 de gauche ne change rien à la valeur : on peut l'omettre.

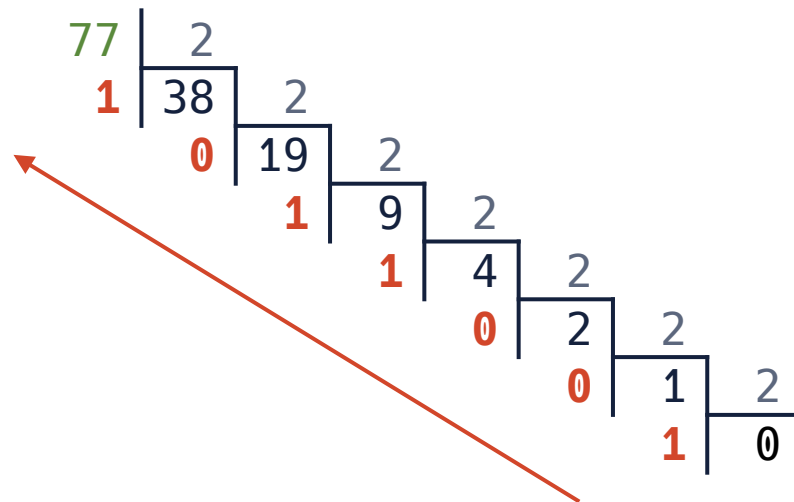
Décimal → binaire : Par divisions successives par 2

2. Binaire ↔ décimal

Convertir **77** en binaire :

1. Diviser par 2, puis diviser le quotient par 2, et ainsi de suite, jusqu'à ce que le **quotient** soit 0.
2. Les **restes** sont les bits : les collecter du bas vers le haut.

Résultat : **77** s'écrit **100'1101** en binaire.



Collecter les restes du bas vers le haut.

Activité · 12 min

Avec la grille de conversion :

1. Convertir **100'0001** en décimal
2. Convertir **1100'1101** en décimal
3. Convertir **131** en binaire
4. Convertir **151** en binaire
5. Convertir **255** en binaire
6. Convertir **0** en binaire
7. Convertir **256** en binaire

Exercices : binaire ↔ décimal

2. Binaire ↔ décimal

Les réponses, dans l'ordre des exercices :

8 bits, soit un octet

128	64	32	16	8	4	2	1	Décimal	Calculs
	1	0	0	0	0	0	1	65	$64 + 1$
1	1	0	0	1	1	0	1	205	$128 + 64 + 8 + 4 + 1$
1	0	0	0	0	0	1	1	131	$128 + 2 + 1$
1	0	0	1	0	1	1	1	151	$128 + 16 + 4 + 2 + 1$
1	1	1	1	1	1	1	1	255	$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1$
							0	0	0

256 ne tient pas sur un octet : il faut un 9^e bit, de poids 256. $(256)_{10} = (1'0000'0000)_2$.

Sommaire

1. Compter

2. Binaire $\leftrightarrow$ décimal

3. Addition

4. Multiplier et diviser

5. Nombres négatifs

6. Hexadécimal

Rappel, en décimal

1. Aligner les deux nombres à droite.
2. Additionner colonne par colonne, en partant de la droite, avec parfois une retenue.

$$\begin{array}{r} 11 \\ 1478 \\ + 365 \\ \hline 1843 \end{array}$$

En binaire, la même méthode

Il suffit de connaître les sommes de deux bits, ou de trois avec une retenue :

$$\begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 10 \\ 1 + 1 + 1 = 11 \end{array}$$

$1 + 1 = 10$: on écrit 0, et on retient 1.

L'addition binaire

3. Addition

1. Écrire les nombres en colonnes, alignés à droite, le plus grand en haut.
2. Additionner colonne par colonne, en partant de la droite.
3. Si la somme d'une colonne dépasse 1, elle s'écrit sur deux bits (**10** ou **11**) : écrire le bit de droite, retenir 1 dans la colonne suivante.
4. Continuer jusqu'à la dernière colonne.
5. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.

Exemple

$$\begin{array}{r} \\ 11011011 \\ + 1110 \\ \hline 11101001 \end{array}$$

Vérification

$$\begin{array}{r} \\ 219 \\ + 14 \\ \hline 233 \end{array}$$

Exercices : addition binaire

3. Addition

Activité · 12 min

Les nombres sont représentés sur 8 bits.

1. Calculer en binaire **1'1101 + 110**
2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
3. Calculer en binaire **1'0111 + 100'0111**
4. Calculer en binaire **1 + 1111'1111**

Les sommes de bits

$$\begin{array}{rcl} 0 + 0 & = & 0 \\ 0 + 1 & = & 1 \\ 1 + 0 & = & 1 \\ 1 + 1 & = & 10 \\ 1 + 1 + 1 & = & 11 \end{array}$$

Exercices : addition binaire

3. Addition

Exercice 1

$$\begin{array}{r} \\ \\ + \\ \hline 1 \end{array}$$

Exercice 2

$$\begin{array}{r} \\ \\ + \\ \hline 3 \end{array}$$

Exercice 3

$$\begin{array}{r} \\ \\ + \\ \hline 1 \end{array}$$

Vérification

$$\begin{array}{r} \\ + \\ \hline 9 \end{array}$$

Le dépassement de capacité

Les nombres sont représentés sur **8 bits**.

3. Addition

$$\begin{array}{r} 11111111 \\ + 11111111 \\ + 1 \\ \hline 100000000 \\ \hline \end{array}$$

8 bits

Vérification en décimal

$$\begin{array}{r} 255 \\ + 1 \\ \hline 0 \text{ ?!} \end{array}$$

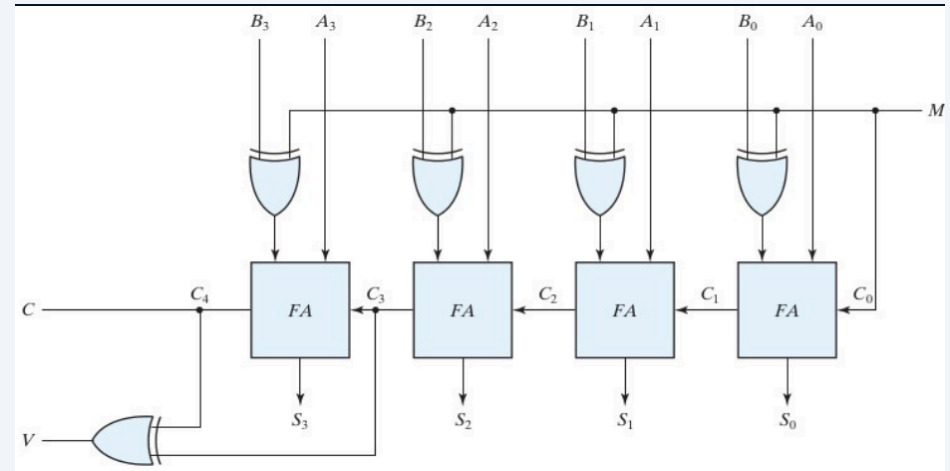
Le bit qui dépasse ne tient pas sur 8 bits : il est perdu. On parle de **dépassement de capacité**.

Un compteur mécanique



Après 99'999 : 00'000.

Un processeur



Un additionneur de 4 bits : la dernière retenue (C) n'entre pas dans le résultat (S_3 à S_0).

Sommaire

1. Compter

2. Binaire $\leftrightarrow$ décimal

3. Addition

4. Multiplier et diviser

5. Nombres négatifs

6. Hexadécimal

Multiplier par des puissances de deux

4. Multiplier et diviser

Par exemple :

$\times 2$: on ajoute un 0 à droite.

Tous les bits se décalent d'une colonne vers la gauche.

$\times 4 = \times 2 \times 2$: on ajoute deux 0 à droite. Les bits se décalent de deux colonnes.

Chaque bit à 1 double son poids : la somme double aussi.

2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0		
256	128	64	32	16	8	4	2	1	Décimal	Calculs
		1	0	0	0	1	1	0	70	$64 + 4 + 2$
	1	0	0	0	1	1	0	0	140	$128 + 8 + 4$
1	0	0	0	1	1	0	0	0	280	$256 + 16 + 8$

$$70 \rightarrow 140 (\times 2) \rightarrow 280 (\times 4)$$

À retenir : multiplier par 2^n , c'est ajouter n zéros à droite. Tous les bits se décalent de n colonnes vers la gauche.

Multiplier par n'importe quel nombre

4. Multiplier et diviser

Rappel, en décimal

$31 = 30 + 1$, donc :

$$1234 \times 31 = 1234 \times 1 + 1234 \times 30$$

$\begin{array}{r} 1234 \\ \times \quad 31 \\ \hline 1234 \\ + 3702 \cdot \\ \hline 38254 \end{array}$	1. 1234×1 2. 1234×30 : 1234×3, décalé d'une colonne (le point remplace le 0) 3. la somme
---	---

En binaire, la même méthode

$5 = 4 + 1$ (5 s'écrit **101**), donc :

$$x \times 5 = x \times 4 + x \times 1$$

Et $x \times 4$, c'est x décalé de deux colonnes. La multiplication devient des décalages suivis d'une simple addition.

À retenir : chaque chiffre du multiplicateur donne une ligne, décalée selon sa position. On additionne les lignes.

$$\begin{array}{r} 111011 \\ \times 110 \\ \hline 0 \\ + 111011 \cdot \\ + 111011 \cdot \cdot \\ \hline 101100010 \end{array}$$

a) $11'1011 \times 0$: la ligne vaut 0.

b) Un point, qui remplace un 0, c) puis $11'1011 \times 1$: on recopie le nombre.

d) Deux points, e) puis $11'1011 \times 1$: on recopie encore.

f) On additionne les lignes.

On peut vérifier :

En passant par le décimal :

$$11'1011 \times 110 = 1'0110'0010$$

$$59 \times 6 = 354$$

Exercices : multiplication binaire

4. Multiplier et diviser

Activité · 8 min

1. Calculer en binaire $1'0101 \times 101$
2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
3. Calculer en binaire $10 \times 10'1010'1101'1101$

Réfléchir avant de se lancer dans un long calcul : il y a une façon rapide de faire.

Exercices : multiplication binaire

4. Multiplier et diviser

1. Calculer en binaire **1'0101** × **101**

$$\begin{array}{r} 10101 \\ \times 101 \\ \hline 10101 \\ + 0 \\ + 10101 \\ \hline 1101001 \end{array}$$

2. Vérifier ce calcul en passant par le décimal.

Binaire	Décimal
1'0101	21
101	5
110'1001	105

21 × **5** = **105** : le calcul binaire est juste.

Exercices : multiplication binaire

4. Multiplier et diviser

3. Calculer en binaire $10 \times 10'1010'1101'1101$

Le long nombre en haut : deux lignes seulement.

$$\begin{array}{r} 10101011011101 \\ \times 10 \\ \hline 0 \\ + 10101011011101 \cdot \\ \hline 101010110111010 \end{array}$$

Plus rapide : 10 vaut 2 , et multiplier par 2 , c'est ajouter un 0 à droite.

$$\begin{aligned} &10'1010'1101'1101 \times 10 \\ &= 101'0101'1011'1010 \end{aligned}$$

Diviser par des puissances de deux

4. Multiplier et diviser

Par exemple :

$\div 2$: on décale les bits d'une colonne vers la droite. Le bit de droite disparaît.

$\div 4 = \div 2 \div 2$: on décale les bits de deux colonnes. Les deux bits de droite disparaissent.

128	64	32	16	8	4	2	1	Décimal	Calculs
1	0	0	0	1	1	0	0	140	$128 + 8 + 4$
	1	0	0	0	1	1	0	70	$64 + 4 + 2$
		1	0	0	0	1	1	35	$32 + 2 + 1$

$$140 \rightarrow 70 (\div 2) \rightarrow 35 (\div 4)$$

À retenir : diviser par 2^n (division entière), c'est enlever les n bits de droite. Tous les bits se décalent de n colonnes vers la droite.

Activité · 8 min

1. Diviser **1'0100** par 2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
2. Diviser **1'0101** par 2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
Y a-t-il une erreur ? Qu'en pensez-vous ?
3. Diviser **1'0000** par 16.
4. Diviser **1111** par 16.

Exercices : division binaire

4. Multiplier et diviser

1. Diviser **1'0100** par 2. Vérifier en passant par le décimal.

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	1	0	0	20	16 + 4
				1	0	1	0	10	8 + 2

2. Diviser **1'0101** par 2. Vérifier en passant par le décimal. Y a-t-il une erreur ?

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	1	0	1	21	16 + 4 + 1
				1	0	1	0	10	8 + 2

Pas d'erreur : la division est entière. Le bit qui disparaît, 1, est le reste.

Exercices : division binaire

4. Multiplier et diviser

3. Diviser **1'0000** par 16.

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	0	0	0	16	16
							1	1	

Les 4 bits qui disparaissent, **0000**, sont le reste : $16 \div 16 = 1$, reste 0.

4. Diviser **1111** par 16.

128	64	32	16	8	4	2	1	Décimal	Calculs
				1	1	1	1	15	$8 + 4 + 2 + 1$
							0	0	

Tous les bits disparaissent : $15 \div 16 = 0$, reste 15.

Pourquoi les divisions successives fonctionnent

4. Multiplier et diviser

On veut convertir $(77)_{10}$ en binaire :

	Décimal	64	32	16	8	4	2	1	Reste
÷ 2	77	1	0	0	1	1	0	1	
÷ 2	38		1	0	0	1	1	0	1
÷ 2	19			1	0	0	1	1	0
÷ 2	9				1	0	0	1	1
÷ 2	4					1	0	0	1
÷ 2	2						1	0	0
÷ 2	1							1	0
÷ 2	0								1

$$(77)_{10} = (100'1101)_2$$

Diviser par 2 décale les bits d'une colonne vers la droite, et le bit qui disparaît est le reste. Les restes sont donc les bits du nombre, de droite à gauche : on les collecte du bas vers le haut.

Sommaire

1. Compter

2. Binaire $\leftrightarrow$ décimal

3. Addition

4. Multiplier et diviser

5. Nombres négatifs

6. Hexadécimal

Qu'est-ce qu'un bit d'information ?

5. Nombres négatifs

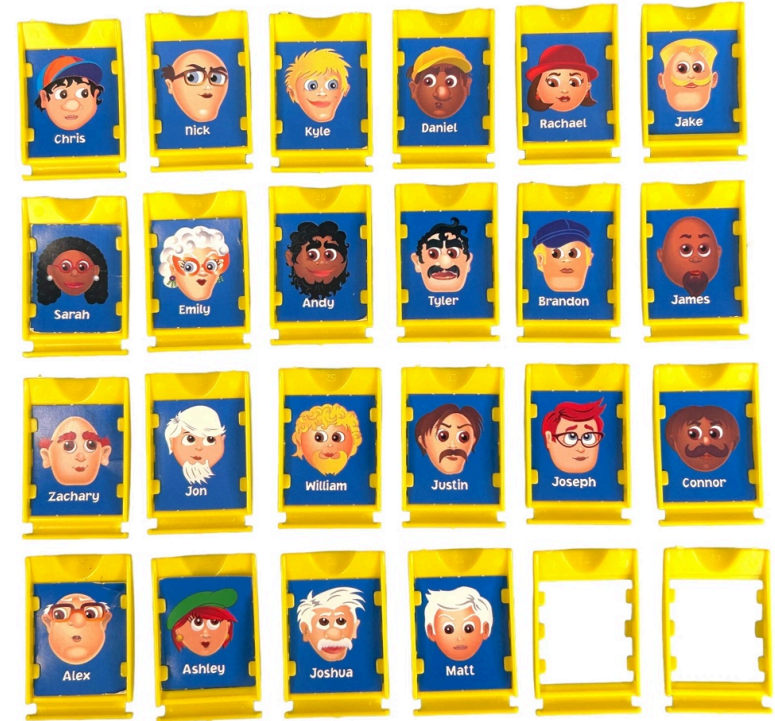
Au jeu « Qui est-ce ? », on pose des questions dont la réponse est **oui** ou **non**.

Chaque réponse donne **1 bit d'information**.

En combinant plusieurs réponses, on trouve le personnage.

Ce nombre est-il négatif ?

Une seule question, oui ou non : 1 bit.



Et les nombres négatifs ?

5. Nombres négatifs

On sait écrire des entiers positifs en binaire : **0**, **111'1000** (120), **1111'1111** (255).

On aimerait aussi écrire des nombres négatifs : -1 , -12 ... Mais un ordinateur n'a que des 0 et des 1, pas de signe $-$.

Pour commencer, et pour simplifier : on n'a que **4 bits** à disposition.

À discuter · 3 min

Quelle idée pourrait-on avoir ?

Première idée : un bit de signe

Le bit de gauche donne le **signe** (1 pour négatif) ; les 3 autres, la **valeur absolue**, le nombre sans son signe.

signe	4	2	1	Décimal
0	0	1	1	3
1	0	1	1	-3
				-4
				7
				0

Activité · 5 min

1. Compléter le tableau.
2. Combien de bits reste-t-il pour la valeur absolue ?
3. Le plus grand nombre ? Le plus petit ?
4. Combien de nombres différents en tout ?

Première idée : un bit de signe

Le bit de gauche donne le **signe** (1 pour négatif) ; les 3 autres, la **valeur absolue**, le nombre sans son signe.

signe	4	2	1	Décimal
0	0	1	1	3
1	0	1	1	-3
1	1	0	0	-4
0	1	1	1	7
0	0	0	0	0
1	0	0	0	0

1. Compléter le tableau : 0 s'écrit 0000 ou 1000.
2. Combien de bits reste-t-il pour la valeur absolue ? 3 bits.
3. Le plus grand nombre ? 0111 = 7. Le plus petit ? 1111 = -7.
4. Combien de nombres différents en tout ? 15 : 16 combinaisons, mais deux pour zéro.

Le bit de signe : deux problèmes

Deux zéros

$$0000 = +0 \quad 1000 = -0$$

Deux combinaisons pour le même nombre :
une est gaspillée.

L'addition ne marche plus

$$\begin{array}{r} 11 \\ 0011 \\ + 1011 \\ \hline 1110 \end{array}$$

3 + (-3) devrait donner 0.
On obtient 1110, soit -6 !

Le complément à deux

La représentation de tous les ordinateurs. Les positifs s'écrivent comme avant, avec un bit de signe à 0. Pour un **négatif** :

exemple : -3

1	Prendre la valeur absolue	3
2	Soustraire 1	2
3	L'écrire en binaire, avec des 0 à gauche pour avoir 4 bits	0010
4	Inverser tous les bits : les 0 deviennent des 1, et inversement	1101

Activité · 4 min

Écrire sur 4 bits :

- -1
- -5

Le bit de signe de **1101** est bien à 1 : c'est un nombre négatif.

Le complément à deux

Écrire sur 4 bits :

-1 $\xrightarrow{\text{valeur absolue}}$ 1 $\xrightarrow{\text{soustraire 1}}$ 0 $\xrightarrow{\text{en binaire, 4 bits}}$ 0000 $\xrightarrow{\text{inverser les bits}}$ 1111

-5 $\xrightarrow{\text{valeur absolue}}$ 5 $\xrightarrow{\text{soustraire 1}}$ 4 $\xrightarrow{\text{en binaire, 4 bits}}$ 0100 $\xrightarrow{\text{inverser les bits}}$ 1011

D'abord, regarder le bit de signe : à 0, le nombre est positif et se convertit comme avant. À 1, il est **néga-tif** :

exemple : 1001

1	Inverser tous les bits	0110
2	Convertir ces bits en décimal	6
3	Ajouter 1	7
4	Mettre un signe – devant	-7

Le complément à deux en résumé

5. Nombres négatifs

Décimal → binaire, pour un nombre négatif



Binaire → décimal, quand le bit de signe est à 1



Cet exemple utilise 4 bits, mais la technique est la même quel que soit le nombre de bits.

Activité · 6 min

Convertir en décimal ces nombres de 4 bits, positifs ou négatifs, en complément à deux :

1. **0110**
2. **1110**
3. **1001**
4. **11**

Rappel, pour un nombre négatif :

1. Inverser tous les bits.
2. Convertir en décimal.
3. Ajouter 1.
4. Mettre un signe –.

Exercices : vers le décimal

5. Nombres négatifs

Convertir en décimal ces nombres de 4 bits, positifs ou négatifs, en complément à deux :

1. **0110** : bit de signe à 0 : positif. Avec la grille : $4 + 2 = 6$.

2. **1110** : bit de signe à 1 : négatif.

1110 $\xrightarrow{\text{inverser les bits}}$ **0001** $\xrightarrow{\text{en décimal}}$ **1** $\xrightarrow{\text{ajouter 1}}$ **2** $\xrightarrow{\text{signe -}}$ **-2**

3. **1001** : bit de signe à 1 : négatif.

1001 $\xrightarrow{\text{inverser les bits}}$ **0110** $\xrightarrow{\text{en décimal}}$ **6** $\xrightarrow{\text{ajouter 1}}$ **7** $\xrightarrow{\text{signe -}}$ **-7**

4. **11** s'écrit **0011** sur 4 bits : bit de signe à 0 : positif. Avec la grille : $2 + 1 = 3$.

Additionner en complément à deux

5. Nombres négatifs

Exemple : $-3 + 5$

$$\begin{array}{r} 1\ 1\ 1 \\ 1\ 1\ 0\ 1 \\ +\ 0\ 1\ 0\ 1 \\ \hline 1\ 0\ 0\ 1\ 0 \\ \hline \end{array}$$

On garde 4 bits :

Le bit qui dépasse est ignoré.

$$1101 = -3$$

$$0101 = 5$$

$$0010 = 2$$

Activité · 8 min

Calculer en binaire, sur 4 bits, puis vérifier en décimal :

1. $-4 + (-2)$
2. $-1 + 1$
3. $-6 + 5$

Convertir les deux nombres en binaire, les additionner, garder 4 bits, puis reconverter le résultat.

Additionner en complément à deux

5. Nombres négatifs

Calculer en binaire, sur 4 bits, puis vérifier en décimal :

1. $-4 + (-2)$

$$\begin{array}{r} 11 \\ 1100 \\ + 1110 \\ \hline 11010 \end{array} \quad \begin{array}{l} 1100 = -4 \\ 1110 = -2 \end{array}$$

4 bits

On garde 4 bits : $1010 = -6$

En décimal : $-4 + (-2) = -6$

2. $-1 + 1$

$$\begin{array}{r} 1111 \\ 1111 \\ + 0001 \\ \hline 10000 \end{array} \quad \begin{array}{l} 1111 = -1 \\ 0001 = 1 \end{array}$$

4 bits

On garde 4 bits : $0000 = 0$

En décimal : $-1 + 1 = 0$

3. $-6 + 5$

$$\begin{array}{r} 1010 \\ + 0101 \\ \hline 1111 \end{array} \quad \begin{array}{l} 1010 = -6 \\ 0101 = 5 \end{array}$$

4 bits

On garde 4 bits : $1111 = -1$

En décimal : $-6 + 5 = -1$

Une nouvelle opération : la soustraction

On sait additionner. On sait écrire les nombres négatifs, et l'addition marche avec eux.

$$x - y = x + (-y)$$

$$\begin{array}{r} \overset{1}{0} \overset{1}{1} \overset{1}{0} \overset{1}{1} \\ + \overset{1}{1} \overset{1}{1} \overset{0}{0} \overset{1}{1} \\ \hline \overset{1}{1} \overset{0}{0} \overset{0}{1} \overset{0}{0} \\ \hline \end{array}$$

4 bits

Exemple : $5 - 3 = 5 + (-3)$

$0101 + 1101$: on garde 4 bits, $0010 = 2$.

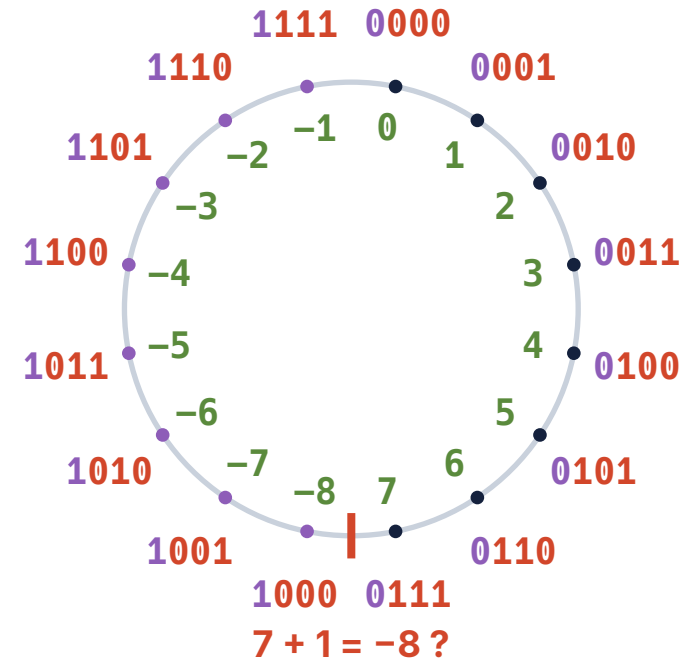
Les nombres de 4 bits

5. Nombres négatifs

signe	4	2	1	Décimal
0	1	1	1	7
1	0	0	1	-7
1	0	0	0	-8
0	0	0	0	0
1	1	1	1	-1

De **-8** à **7** : **16 nombres** différents, avec un seul zéro.

Sur la roue, ajouter 1 avance d'un cran dans le sens des aiguilles d'une montre.



Sur 8 bits, et plus

5. Nombres négatifs

Jusqu'ici, 4 bits pour simplifier ; en réalité, on utilise toujours au moins 8 bits.

signe	64	32	16	8	4	2	1	Décimal
0	1	1	1	1	1	1	1	127
0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	-1
1	0	0	0	0	0	0	0	-128

Bits	De	à
4	-8	7
8	-128	127
16	-32'768	32'767
32	-2'147'483'648	2'147'483'647

Les mêmes bits, deux interprétations

```
unsigned char u = 127;
signed char   s = 127;

u = u + 1;
s = s + 1;
// u vaut 128, s vaut -128
```

u est un nombre sans signe, toujours positif ; **s** est un nombre signé. Tous deux ont 8 bits.

	Bits	u	s
	0111'1111	127	127
+ 1	1000'0000	128	-128
	0000'0000	0	0
- 1	1111'1111	255	-1

Mêmes opérations, mêmes bits : seule l'interprétation en décimal change.

Sommaire

1. Compter

2. Binaire $\leftrightarrow$ décimal

3. Addition

4. Multiplier et diviser

5. Nombres négatifs

6. Hexadécimal

À quoi sert l'hexadécimal ?

6. Hexadécimal

Décimal	Binaire	Hexadécimal
0	0	0
9	1001	9
15	1111	F
16	1'0000	10
27	1'1011	1B
255	1111'1111	FF
256	1'0000'0000	100
200'000	11'0000'1101'0100'0000	30D40

L'hexadécimal écrit les nombres binaires de façon **compacte**.

Deux symboles hexadécimaux suffisent pour un octet : $(1111'1111)_2 = (FF)_{16}$.

- Compter en hexadécimal, c'est compter en **base 16**, avec 16 symboles :

0 1 2 3 4 5 6 7 8 9 A B C D E F

- Un symbole représente les nombres de 0 à 15, soit **4 bits**.
- On combine les symboles pour écrire de plus grands nombres : **4E**, **F0AF**, **109A22**.

Préciser la base

6. Hexadécimal

Nous avons maintenant trois façons d'écrire les nombres : binaire (base 2), décimal (base 10), hexadécimal (base 16).

Pour préciser la base, on écrit le nombre entre parenthèses et la base en bas à droite :

Binaire

$(111)_2$ $(1011010)_2$

Décimal

$(111)_{10}$ $(89)_{10}$

Hexadécimal

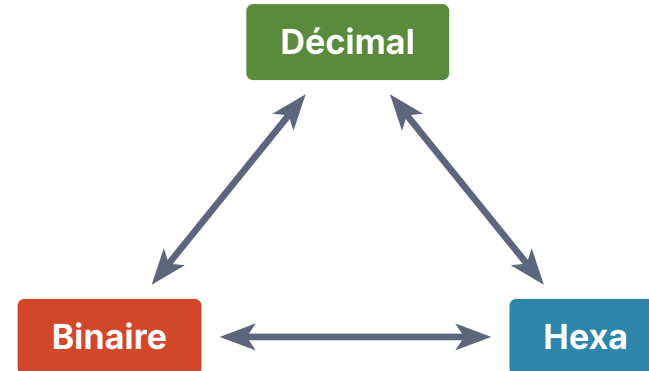
$(111)_{16}$ $(89)_{16}$ $(90F)_{16}$
 $(CAFEBAD)_{16}$

$$(111)_2 = (7)_{10} \quad (111)_{16} = (273)_{10}$$

Les 16 symboles

Binaire	Hexa	Décimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Les 16 symboles hexadécimaux correspondent aux 16 nombres binaires de 4 bits, et aux nombres décimaux de 0 à 15.



Ce tableau permet de convertir immédiatement les petits nombres entre les trois bases.

Hexadécimal ↔ binaire

6. Hexadécimal

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Au-delà de **F**, la conversion reste simple : chaque groupe de 4 bits donne un symbole.

1. Couper le binaire en groupes de 4 bits, **en partant de la droite**.
2. Remplacer chaque groupe par son symbole, avec le tableau.

Dans l'autre sens : chaque symbole donne 4 bits.

Cette technique permet de convertir très facilement dans les deux sens :

- de **binaire** à **hexadécimal**,
- d'**hexadécimal** à **binaire**.

1111 0011
F 3
11 1111
3 F
11 0000 1011 1110
3 0 B E

Décimal

Binaire



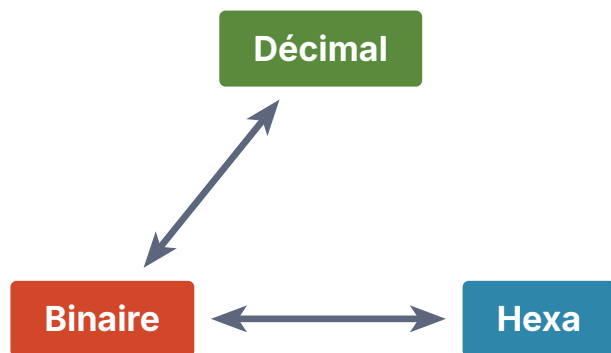
Hexa

Décimal ↔ hexadécimal, en passant par le binaire

On sait convertir entre décimal et binaire, et entre binaire et hexadécimal.

En enchaînant les deux, on convertit en passant par le binaire :

- de décimal à hexadécimal,
- d'hexadécimal à décimal.



$$\begin{array}{r} 243 \\ 1111 \ 0011 \\ F \ 3 \\ (243)_{10} = (F3)_{16} \end{array}$$
$$\begin{array}{r} 63 \\ 11 \ 1111 \\ 3 \ F \\ (63)_{10} = (3F)_{16} \end{array}$$
$$\begin{array}{r} 12'478 \\ 11 \ 0000 \ 1011 \ 1110 \\ 3 \ 0 \ B \ E \\ (12'478)_{10} = (30BE)_{16} \end{array}$$

Les nombres négatifs en hexadécimal

6. Hexadécimal

On passe par le binaire, en complément à deux, puis on groupe les bits par 4.

Sur 8 bits

Complément à deux de -4

$$\begin{array}{cc} & -4 \\ \text{1111} & \text{1100} \\ \text{F} & \text{C} \\ (-4)_{10} = (\text{FC})_{16} \end{array}$$

Nombre positif

$$\begin{array}{cc} & 127 \\ \text{0111} & \text{1111} \\ \text{7} & \text{F} \\ (127)_{10} = (\text{7F})_{16} \end{array}$$

Sur 16 bits

Complément à deux de -29'672

$$\begin{array}{cccc} & & -29'672 \\ \text{1000} & \text{1100} & \text{0001} & \text{1000} \\ \text{8} & \text{C} & \text{1} & \text{8} \\ (-29'672)_{10} = (\text{8C18})_{16} \end{array}$$

Nombre positif

$$\begin{array}{cccc} & & 31'768 \\ \text{0111} & \text{1100} & \text{0001} & \text{1000} \\ \text{7} & \text{C} & \text{1} & \text{8} \\ (31'768)_{10} = (\text{7C18})_{16} \end{array}$$

Hexadécimal → décimal, directement

6. Hexadécimal

Hexa	Décimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

On utilise une grille dont les poids sont des puissances de 16 : 1, 16, 256, 4'096, 65'536...

1. Écrire le nombre dans la grille, en partant de la droite.
2. Convertir chaque symbole en sa valeur décimale, avec le tableau.
3. Multiplier chaque valeur par son poids, et additionner.

Exemple : convertir $(90F)_{16}$ en décimal

Poids	4'096	256	16	1
Symbole		9	0	F
Valeur		9	0	15

Décimal

Binaire

Hexa

$$9 \times 256 + 0 \times 16 + 15 \times 1 = 2'319$$

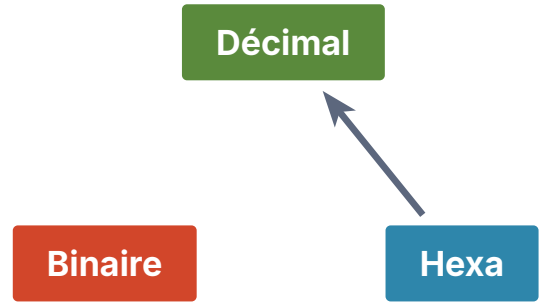
$$(90F)_{16} = (2'319)_{10}$$

Hexadécimal → décimal : un 2e exemple

Hexa	Décimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Convertir $(B33F)_{16}$ en décimal :

Poids	4'096	256	16	1
Symbole	B	3	3	F
Valeur	11	3	3	15



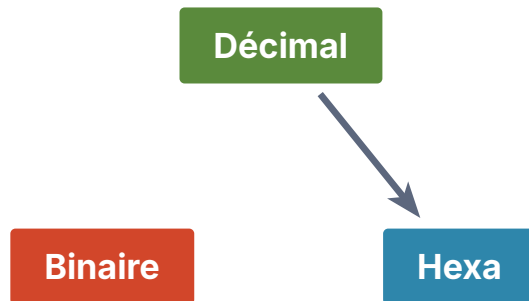
$$11 \times 4'096 + 3 \times 256 + 3 \times 16 + 15 \times 1 = 45'887$$

$$(B33F)_{16} = (45'887)_{10}$$

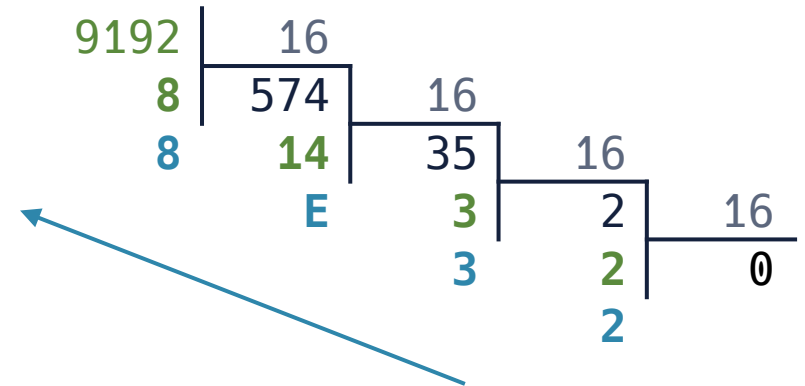
Décimal → hexadécimal, directement

1. Divisions **entières** successives par 16, jusqu'à ce que le **quotient** soit 0.
2. Convertir chaque **reste**, de 0 à 15, en son **symbole** hexadécimal.
3. Collecter les symboles du bas vers le haut.

Résultat : $(9'192)_{10} = (23E8)_{16}$



Exemple : convertir $(9'192)_{10}$ en hexadécimal



Collecter les symboles du bas vers le haut.

Exercices : hexadécimal

6. Hexadécimal

Binaire	Hexa	Décimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Activité · 15 min

Avec les deux tableaux :

- 1) Écrire **70** en hexadécimal.
- 2) Convertir **(B7)₁₆** en décimal :
 - a) d'abord en passant par le binaire ;
 - b) puis directement, avec la grille des poids.
- 3) Combien de symboles hexadécimaux faut-il pour écrire 8 bits, c'est-à-dire les nombres de 0 à 255 ?

Par exemple, **(E)₁₆** utilise 1 symbole, **(ACAD)₁₆** en utilise 4.

Poids	4'096	256	16	1
Symbole				
Valeur				

Exercices : hexadécimal

6. Hexadécimal

1. Écrire **70** en hexadécimal.

8 bits, soit un octet

128	64	32	16	8	4	2	1
0	1	0	0	0	1	1	0

Décimal	Calculs
70	64 + 4 + 2

0100 0110
4 6

Résultat : **(70)₁₀** = **(46)₁₆**

Exercices : hexadécimal

2. Convertir $(B7)_{16}$ en décimal.

a) En passant par le binaire : chaque symbole donne 4 bits.

B **7**
1011 **0111**

8 bits, soit un octet

128	64	32	16	8	4	2	1
1	0	1	1	0	1	1	1

Décimal	Calculs
183	$128 + 32 + 16 + 4 + 2 + 1$

b) Directement, avec la grille des poids :

Poids	4'096	256	16	1
Symbole			B	7
Valeur			11	7

$$11 \times 16 + 7 \times 1 = 183$$

$$(B7)_{16} = (183)_{10}$$

Exercices : hexadécimal

6. Hexadécimal

3. Combien de symboles hexadécimaux faut-il pour écrire 8 bits, c'est-à-dire les nombres de 0 à 255 ?

Par la définition

- Il faut **8 bits** pour écrire les nombres de 0 à 255.
- Un symbole hexadécimal représente **4 bits**.
- Deux symboles représentent donc 8 bits.

Par le calcul

Le plus petit et le plus grand nombre de deux symboles :

Poids	16	1
Symbole	0	0
Valeur	0	0

$$0 \times 16 + 0 \times 1 = 0$$

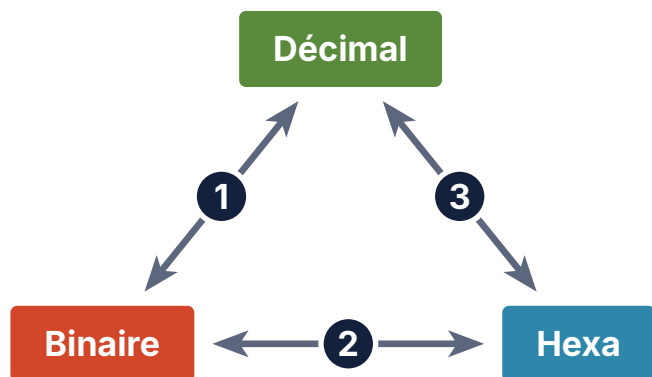
Poids	16	1
Symbole	F	F
Valeur	15	15

$$15 \times 16 + 15 \times 1 = 255$$

Réponse : **2 symboles**, de $(00)_{16} = (0)_{10}$ à $(FF)_{16} = (255)_{10}$.

Récapitulatif : convertir entre les trois bases

6. Hexadécimal



Un convertisseur entre les trois bases :
ri.nwolff.info/binary-converter

1 Binaire ↔ décimal

→ **décimal** : additionner les poids des bits à 1

→ **binaire** : soustractions successives, ou divisions successives par 2

2 Binaire ↔ hexadécimal

Par groupes de 4 bits, en utilisant le tableau des 16 symboles.

3 Hexadécimal ↔ décimal

→ **décimal** : chaque symbole en valeur, multipliée par sa puissance de 16

→ **hexadécimal** : divisions successives par 16, chaque reste en symbole

Contenu et outils

Convertisseur entre binaire, décimal et hexadécimal

ri.nwolff.info/binary-convert

The Almighty Guru – le nombre 255 dans les jeux vidéo

www.thealmightyguru.com/Wiki/index.php?title=255_in_video_games

Images

Konstantin Lanzet – une mémoire à tores magnétiques (CC BY-SA 3.0)

commons.wikimedia.org/wiki/File:KL_CoreMemory_Macro.jpg

Geoffrey Saxe – le comptage des Oksapmin

culturecognition.com/new-page-3

Hellbus – un compteur kilométrique (domaine public)

commons.wikimedia.org/wiki/File:Odometer_rollover.jpg

**M. Morris Mano et Michael D. Ciletti – *Digital Design*,
l'additionneur-soustracteur de 4 bits**

Hasbro – le jeu « Qui est-ce ? »